
EXECUTION PLAN

Aura Living

A Sprint-by-Sprint Build Plan for the Aura Living Frontend
— Companion to the v1.1 Architecture Plan

Brand: Aura Living — Light, Life, and Living Beauty

Niche: Premium Home Decor (Lamps, Plants, Candles)

Market: Islamic Republic of Pakistan

Structure: 5 Phases · 11 Sprints · 9 Weeks

Pause Points: 11 (5 major checkpoints)

Version: 1.0 | June 2026

Aura Living · PREMIUM HOME DECOR EXECUTION
PLAN · v1.0

ABSTRACT

This document is the execution plan for building the Aura Living frontend described in the v1.1 Frontend Architecture Plan. Where the architecture plan specifies what to build and why, this execution plan specifies the order in which to build it, the deliverables that mark each sprint complete, and the decision gates at which the build pauses for review. The plan preserves the five-phase, nine-week structure from Chapter 14 of the architecture plan and decomposes each phase into specific sprints: four sprints in Phase 1 (Foundation), five sprints in Phase 2 (Core Pages), two sprints in Phase 3 (Animations), two sprints in Phase 4 (SEO and Performance), and two sprints in Phase 5 (Pre-Launch QA), for a total of eleven sprints.

Each sprint follows the same structure: a single-sentence goal, a numbered task list with concrete actions, a files-created count with key examples, binary acceptance criteria, and a pause point (where applicable) at which the build stops for stakeholder sign-off. Eleven pause points are placed at the moments where course correction is cheapest: after the scaffold, after the design tokens, after each major page, and at the end of each phase. Five of these are major checkpoints (Phase 1 sign-off, Phase 2 sign-off, Phase 3 sign-off, Phase 4 sign-off, and GO LIVE) that warrant a live walkthrough rather than async review.

The plan also documents six prerequisites that must be in place before Sprint 1.1 begins (GitHub repo, Vercel account, domain, image assets, brand wordmark, architecture plan access) and a Definition of Done checklist for each of the five phases. This document is the operational counterpart to the architecture plan and should be read alongside it; every sprint references the architecture plan by chapter number for the underlying decisions.

TABLE OF CONTENTS

1. Overview and How to Use This Plan 2

1.1 How Each Sprint Is Structured 2

1.2 The Pause-Point Protocol 3

1.3 Prerequisites 3

2. Phase 1 — Foundation (Weeks 1-2) 4

2.1 Sprint 1.1 — Scaffold and Tooling (Days 1-2) 4

Tasks 4

2.2 Sprint 1.2 — Design Tokens and Global Styles (Days 3-4) 5

Tasks 6

2.3 Sprint 1.3 — Routing Shell and Placeholders (Days 5-6) 7

Tasks 7

2.4 Sprint 1.4 — CI/CD Pipeline and Staging Deploy (Days 7-10) 8

Tasks 8

3. Phase 2 — Core Pages (Weeks 3-5) 10

3.1 Sprint 2.1 — Mock Data and Service Layer (Days 11-12) 10

Tasks 10

3.2 Sprint 2.2 — Layout Primitives and Shell (Days 13-15) 11

Tasks 12

3.3 Sprint 2.3 — Homepage (Days 16-18) 13

Tasks 13

3.4 Sprint 2.4 — PLP and PDP (Days 19-23) 15

Tasks 15

3.5 Sprint 2.5 — Cart, Checkout, and Confirmation (Days 24-28) 17

Tasks 17

4. Phase 3 — Polish and Animations (Weeks 6-7) 18

4.1 Sprint 3.1 — Smooth Scroll and Parallax Infrastructure (Days 29-31) 19

Tasks 19

4.2 Sprint 3.2 — Page-Level Choreography (Days 32-36) 20

Tasks 21

5. Phase 4 — SEO and Performance (Week 8) 22

5.1 Sprint 4.1 — Metadata, Schema, and Sitemap (Days 37-39) 23

Tasks 23

5.2 Sprint 4.2 — Performance Audit and Bundle Tuning (Days 40-42) 25

Tasks 25

6. Phase 5 — Pre-Launch QA (Week 9) 27

6.1 Sprint 5.1 — Test Suite Hardening (Days 43-45) 27

Tasks 27

6.2 Sprint 5.2 — Content, Legal, and Soft Launch (Days 46-48) 28

Tasks 29

7. Pause Points and Decision Gates 30

7.1 Reviewer Response Times 30

7.2 Escalation at Major Checkpoints 31

8. Open Questions and Prerequisites 31

8.1 GitHub Repository 31

8.2 Domain Registration 32

8.3 Vercel Account 32

<u>8.4 Image Assets</u>	<u>32</u>
<u>8.5 Brand Wordmark</u>	<u>33</u>
<u>8.6 Architecture Plan Access</u>	<u>33</u>
<u>8.7 Scope for First Build Session</u>	<u>33</u>

[9. Definition of Done per Phase](#) [34](#)

<u>9.1 Phase 1 — Foundation Definition of Done</u>	<u>34</u>
<u>9.2 Phase 2 — Core Pages Definition of Done</u>	<u>36</u>
<u>9.3 Phase 3 — Animations Definition of Done</u>	<u>38</u>
<u>9.4 Phase 4 — SEO and Performance Definition of Done</u>	<u>40</u>
<u>9.5 Phase 5 — Pre-Launch QA Definition of Done</u>	<u>43</u>
<u>9.6 Document Version Note</u>	<u>45</u>

Note: This Table of Contents is generated via field codes. To refresh page numbers after editing, right-click the TOC and select "Update Field" then "Update entire table."

1. Overview and How to Use This Plan

This document is the execution plan for building the Aura Living frontend described in the v1.1 Frontend Architecture Plan. Where the architecture plan specifies what to build and why, this execution plan specifies the order in which to build it, the deliverables that mark each sprint complete, and the decision gates at which the build pauses for review. It is the operational counterpart to the architecture document and should be read alongside it.

The plan preserves the five-phase, nine-week structure defined in Chapter 14 of the architecture plan but decomposes each phase into specific sprints with concrete file lists, acceptance criteria, and pause points. Each sprint is sized at two to four working days for a single full-time frontend engineer. With two engineers working in parallel on independent sprints, the timeline can be compressed to roughly six weeks; with one engineer part-time, it expands to twelve. The phase sequence should not be reordered regardless of team size — the design system must exist before the pages that consume it, and the pages must exist before the animations that choreograph them.

1.1 How Each Sprint Is Structured

Every sprint in this document follows the same structure for quick scanning. The Goal is a single sentence stating what the sprint delivers. The Tasks section is a numbered list of concrete actions,

each small enough to be a single commit. The Files Created section lists the approximate count and the key examples, so the engineer can estimate the scope before starting. The Acceptance Criteria section is the binary test: either the criteria are met or the sprint is not done. The Pause Point section, where present, marks the gates at which the build stops for stakeholder review before the next sprint begins.

Acceptance criteria are deliberately concrete and verifiable. A criterion like 'the homepage looks good' is useless because it cannot be tested; a criterion like 'the homepage renders all six sections from Chapter 7.1 with correct typography and no console warnings' can be tested by opening the page and checking. Engineers should treat the acceptance criteria as their definition of done for each sprint.

1.2 The Pause-Point Protocol

There are eleven pause points in this plan, marked with the « symbol. At each pause point, the engineer stops work, posts a summary of what was completed, and waits for stakeholder sign-off before proceeding. This is not bureaucracy; it is risk management. The cost of a one-day pause for review is dramatically lower than the cost of building three sprints on a wrong assumption and having to redo them. The pause points are placed at the moments where course correction is cheapest: after the scaffold, after the design tokens, after each major page, and at the end of each phase.

Stakeholders should respond to a pause point within one business day. If no response is received, the engineer may proceed at their own risk, but the assumption is that silence is consent, not that the work is paused indefinitely. If a longer review period is needed (a senior developer on vacation, a brand director travelling), the engineer should be told in advance so they can either pick up an independent sprint or pause cleanly.

1.3 Prerequisites

Before Sprint 1.1 begins, six prerequisites must be in place. These are listed in detail in Chapter 8 (Open Questions and Prerequisites) and summarised here: a GitHub repository for the code, a Vercel account for hosting, the production domain (auraliving.pk or a placeholder), a brand wordmark (or written approval to use a typographic placeholder), a source for placeholder imagery (Pexels is the default), and access to the v1.1 architecture plan as the reference document. If any of these are missing, Sprint 1.1 cannot start; the engineer should escalate before day one.

Phase	Weeks	Sprints	Primary Deliverable
1. Foundation	1-2	1.1 → 1.4 (4 sprints)	Runnable project on staging with CI/CD
2. Core Pages	3-5	2.1 → 2.5 (5 sprints)	Full purchase journey on mock data
3. Animations	6-7	3.1 → 3.2 (2 sprints)	Premium feel, reduced-motion safe
4. SEO & Perf	8	4.1 → 4.2 (2 sprints)	Lighthouse 90+, CWV green, schema valid
5. Pre-Launch QA	9	5.1 → 5.2 (2 sprints)	Soft launch to production

Table 1.1 — Five-phase execution structure

2. Phase 1 — Foundation (Weeks 1-2)

Phase 1 establishes the technical foundation on which every subsequent sprint depends. The deliverable at the end of Phase 1 is a runnable Next.js 16 project deployed to staging with all 27 routes reachable, the design system implemented, and the CI/CD pipeline green on every pull request. Phase 1 is unglamorous — there is nothing to show stakeholders at the end of week one — but skipping or rushing it compounds problems in every subsequent phase. A misconfigured TypeScript strict mode in week one becomes a hundred suppressed errors by week five.

2.1 Sprint 1.1 — Scaffold and Tooling (Days 1-2)

Goal: A runnable Next.js 16 project on localhost:3000 with the full toolchain configured and zero custom code beyond the scaffold. This sprint is purely mechanical; no design decisions are made here. The engineer runs the official create-next-app, installs the dependencies listed in Appendix B of the architecture plan, configures the linters, and verifies the dev server starts clean.

Tasks

- Run `pnpm create next-app@latest aura-living` with: TypeScript strict, Tailwind CSS 4, ESLint, App Router, `src/` layout, import alias `@/*`.

- Install runtime dependencies per Appendix B: zustand, react-hook-form, zod, @hookform/resolvers, framer-motion (motion package), gsap, @gsap/react, lenis, next-intl, @tanstack/react-query, clsx, tailwind-merge, lucide-react.
- Install dev dependencies: vitest, @testing-library/react, @testing-library/jest-dom, @playwright/test, msw, @axe-core/playwright, size-limit, lint-staged, husky.
- Create the folder skeleton per Chapter 4 of the architecture plan: app/, features/, components/, lib/, services/, utils/, styles/, tests/.
- Initialise Husky pre-commit hook: ESLint + Prettier + tsc --noEmit on staged files.
- Add .nvmrc and .tool-versions for Node 20 LTS pinning.

Files created: approximately 15 (configuration files and folder skeleton). The key files are package.json, tsconfig.json, next.config.ts, tailwind.config.ts, .eslintrc, .prettierrc, .nvmrc, and the empty folder structure.

Acceptance criteria: pnpm dev starts without errors on localhost:3000; pnpm build produces a successful production build with no warnings; pnpm test runs an empty test suite and exits cleanly; pnpm lint passes; the pre-commit hook fires on a test commit and blocks an intentionally broken file.

» Pause Point 1 — Verify scaffold and dependency versions look correct before design tokens are built on top of them. Reviewer should check: Node version, pnpm version, all dependency versions match Appendix B, the folder structure matches Chapter 4, the dev server loads the default Next.js welcome page.

2.2 Sprint 1.2 — Design Tokens and Global Styles (Days 3-4)

Goal: The gold-on-black design system is live in code, every colour and spacing value comes from a CSS custom property, and no inline styles are permitted anywhere in the codebase. This sprint implements the visual foundation specified in Chapter 5 and extended in Chapter 23 of the architecture plan.

Tasks

- Write styles/tokens.css with the full design token set: IG-1 Ink Gold palette (bg, primary, accent, body.heading, body.body, body.muted, body.surfaceLight, body.surfaceMid, body.border), fluid type scale, 8px spacing grid, radii, shadows, z-index scale (Ch.23.1), breakpoint scale (Ch.23.2), cubic-bezier easing curves.

- Configure `tailwind.config.ts` to consume tokens via `@theme inline` so utilities like `bg-surface-light` and `z-modal` resolve to the CSS custom properties.
- Write `styles/globals.css` with the CSS reset, body font defaults, base element styles, the focus-visible ring (gold, 2px offset 2px), and the prefers-reduced-motion global guard.
- Load fonts via `next/font/google`: Inter (body sans, Latin), Fraunces (display serif, Latin), Noto Nastaliq Urdu (RTL-only, scoped to `[dir='ur']` selector).
- Add a custom ESLint rule (or plugin) that fails the build on any `style={ { }}` attribute — this enforces the no-inline-styles rule from Chapter 5.1.
- Create `lib/cn.ts` — the `clsx` + `tailwind-merge` helper that every component uses for conditional class composition.

Files created: approximately 8. The key files are `styles/tokens.css`, `styles/globals.css`, `tailwind.config.ts`, `lib/cn.ts`, `app/fonts.ts`.

Acceptance criteria: a throwaway page renders a gold heading on a cream background with no console warnings; the focus-visible ring appears on tab navigation; the ESLint rule catches a deliberately inlined style and fails the build; the Inter and Fraunces fonts load via `next/font` with no layout shift; the total font payload is under 80KB.

|| Pause Point 2 — Eyeball the design tokens against the architecture plan. Reviewer should check: the gold (#C9A84C) renders correctly on black (#0E0E0E); the dark gold (#8A6B26) meets WCAG AA on white; the type scale feels balanced at mobile and desktop widths; the spacing scale produces consistent rhythm. This is the moment to request colour or spacing tweaks — they are cheap now, expensive after pages are built.

2.3 Sprint 1.3 — Routing Shell and Placeholders (Days 5-6)

Goal: All 27 routes from the route taxonomy (Table 3.2, updated in v1.1) are reachable, each rendering a placeholder page with the correct metadata and locale prefix. The routing shell is the skeleton on which Phase 2 hangs the page bodies.

Tasks

- Create the `app/[locale]/` directory structure with a root `layout.tsx` that wraps every locale route.
- Stub all 27 routes from Table 3.2: homepage, shop, shop/[category], shop/[category]/[subcategory], product/[slug], collections/[slug], cart, checkout, account, account/orders,

account/addresses, account/wishlist, account/settings, lookbook, lookbook/[slug], journal, journal/[slug], about, contact, faq, shipping-returns, privacy, terms, search, orders/[id]/confirmed, accessibility-statement, maintenance, 404.

- Each stub renders an h1 with the route name and 'Coming soon' copy, wrapped in the root layout.
- Add middleware.ts with next-intl locale routing: en and ur prefixes, Accept-Language detection, root URL redirect to preferred locale, exclusion of /api, /_next, and static files from locale handling.
- Add app/not-found.tsx (basic 404, full version comes in Phase 2) and app/error.tsx (basic error boundary, full version comes in Phase 3).
- Add a temporary navigation menu (deleted in Sprint 2.2 when the real Header ships) listing all 27 routes for click-testing.

Files created: approximately 32. The bulk is the per-route page.tsx stubs, plus the layout, middleware, not-found, and error boundaries.

Acceptance criteria: manual click-test of all 27 routes from the temp nav menu produces no 404s on declared routes; locale switch from English to Urdu flips the dir attribute on the html element; the URL / (without locale prefix) redirects to /en; the URL /nonexistent returns the 404 page; the URL /en/nonexistent also returns the 404 page (not a server error).

» Pause Point 3 — Confirm the route structure matches the architecture plan. Reviewer should check: all 27 routes are present; the locale prefixing works in both directions; the [locale] segment does not break any static-asset routes; the not-found and error boundaries render in both locales. This is the last chance to add or remove routes cheaply.

2.4 Sprint 1.4 — CI/CD Pipeline and Staging Deploy (Days 7-10)

Goal: Every pull request auto-deploys to a Vercel preview URL; every merge to main deploys to staging. The CI pipeline runs four quality gates on every PR and blocks merge on any failure. This sprint closes Phase 1 and produces the first deployable artifact for stakeholder review.

Tasks

- Create the GitHub repository aura-living (private) and push the scaffold from Sprints 1.1-1.3 as the initial commit on main.

- Wire the Vercel project: production branch = main, preview = all PR branches, build command = pnpm build, output directory = .next (auto-detected).
- Add environment variables in Vercel per Chapter 13.1: NEXT_PUBLIC_SITE_URL, NEXT_PUBLIC_USE MOCKS=true, NEXT_PUBLIC_LOCALES=en,ur, NEXT_PUBLIC_DEFAULT_LOCALE=en, plus placeholder slots for future Supabase and Algolia keys (empty for now).
- Add .github/workflows/ci.yml per Chapter 13.4 and Chapter 16.6 with four jobs: lint-typecheck (ESLint + tsc --noEmit, ~45s), unit-component (Vitest run with --coverage, ~35s), e2e-essentials (Playwright with @critical tag, initially empty set, ~3m 20s), bundle-size (size-limit against the 130KB First Load JS budget, ~30s).
- Add .github/pull_request_template.md per Chapter 28.2 with sections: Why, What, Testing, Risk, Rollback, Screenshots, Checklist.
- Add docs/adr/ directory with ADR-0001 (use Next.js 16 App Router), ADR-0002 (use Zustand over Redux for cart), ADR-0003 (use next-intl over i18next).
- Open PR #1: 'feat: initial scaffold + design tokens + routes + CI'. Merge to main. Confirm staging deployment goes live.
- Open PR #2: a tiny copy edit on the homepage placeholder. Confirm the PR preview deployment goes live within 90 seconds and the CI pipeline runs all four gates.

Files created: approximately 6 (CI YAML, PR template, env example, README, three ADRs, Vercel config).

Acceptance criteria: the staging URL (staging.auraliving.pk or the Vercel-generated subdomain) loads the homepage placeholder with the correct fonts and colours; PR #2 auto-deploys a preview URL within 90 seconds; the CI pipeline runs all four jobs on PR #2 and they all pass; merging PR #2 to main triggers a staging redeploy; the Vercel dashboard shows preview deployments for both PRs.

|| Pause Point 4 — ★ PHASE 1 MAJOR CHECKPOINT.
Reviewer should: visit the staging URL and click through all 27 routes; verify the locale switch works; check that the CI pipeline ran all four gates on the test PR; confirm the Vercel preview deployment works; verify the env variables are set correctly. Once Phase 1 is signed off, Phase 2 can begin. Phase 1 is the foundation — if anything is wrong here, fix it before proceeding.

3. Phase 2 — Core Pages (Weeks 3-5)

Phase 2 builds the core customer journey: homepage, product listing page, product detail page, cart, checkout, and order confirmation. By the end of Phase 2, a user can complete the full purchase journey from homepage to order confirmation using mock data, with no animations beyond basic hover states. Phase 2 is the longest phase (five sprints over three weeks) because every page in the customer journey is built here, and each page touches multiple systems (data fetching, state management, forms, validation, accessibility).

The sprints in Phase 2 are sequenced for dependency resolution. The mock data and service layer (Sprint 2.1) must exist before any page can fetch data. The layout primitives and shell (Sprint 2.2) must exist before any page can be composed. The homepage (Sprint 2.3) is built next as the highest-visibility page and the first to be reviewed by stakeholders. The PLP and PDP (Sprint 2.4) are the conversion path. The cart, checkout, and confirmation (Sprint 2.5) close the loop. Reordering these sprints would create artificial blockers.

3.1 Sprint 2.1 — Mock Data and Service Layer (Days 11-12)

Goal: All UI in subsequent sprints can pull realistic mock data through the typed service interface, with MSW intercepting the same calls in tests. This sprint builds the data foundation that every page in Phase 2 will consume.

Tasks

- Author services/mocks/data/ with realistic mock content: 48 SKUs (16 lamps, 16 plants, 16 candles) with real-feeling names ('Brass Lotus Lamp', 'Monstera Deliciosa — Medium', 'Saffron and Oud Candle'), PKR prices (1,999 to 24,999), high-quality Pexels image URLs, full metadata (dimensions, materials, care instructions).
- Define TypeScript contracts in services/[domain]/types.ts for each domain: product, cart, checkout, search, review, journal, collection, account, inventory. Each contract is the interface that both the mock and the future Supabase implementation must satisfy (Chapter 29.4).
- Implement the mock services in services/[domain]/mock.ts with a 250-600ms artificial delay to simulate real network latency — fast enough for development, slow enough to expose loading states.
- Wire the service resolver in services/[domain]/index.ts to switch on the NEXT_PUBLIC_USE MOCKS environment variable. Components import the resolved service and never know whether they are talking to a mock or a real backend.

- Add MSW handlers in tests/mocks/handlers/ that mirror the service contracts. Each handler returns the same data shape as the mock service, so component tests can run against MSW instead of against the mock service directly.
- Write unit tests for the price formatter, slug generator, and cart-total calculator — the three pure functions that every page depends on.

Files created: approximately 25. The key files are services/mocks/data/products.json (48 SKUs), services/product/types.ts, services/product/mock.ts, services/product/index.ts, services/cart/types.ts, services/cart/mock.ts, tests/mocks/handlers/products.ts, tests/mocks/handlers/cart.ts.

Acceptance criteria: a Vitest test calling `productService.list({ category: 'lamps' })` returns 16 products with the correct TypeScript shape; an MSW-intercepted component test renders a `ProductCard` with the same data; the artificial delay is observable in the test (the test waits ~400ms); the unit tests for the three pure functions pass.

3.2 Sprint 2.2 — Layout Primitives and Shell (Days 13-15)

Goal: The site shell — header, footer, mobile menu, cart drawer, locale switcher — is fully functional and composes correctly on every route. This sprint builds the components that wrap every page in Phase 2 and beyond.

Tasks

- Build components/layout/Header.tsx — sticky on scroll, gold-on-black, with logo (typographic placeholder), primary nav, search trigger, cart trigger with count badge, locale switcher, account menu.
- Build components/layout/Footer.tsx — link columns (shop, about, help, legal), payment method icons (COD, JazzCash, Easypaisa, Visa, Mastercard), accessibility-statement link, social links, copyright.
- Build components/layout/MobileMenu.tsx — full-screen sheet with accordion nav, opened from a hamburger button in the header, closed via swipe-down or X button.
- Build features/cart/CartDrawer.tsx — full-screen sheet on mobile, side drawer on desktop, opened from the header cart trigger, shows line items with qty controls and subtotal.
- Build features/i18n/LocaleSwitcher.tsx — globe icon with a dropdown showing English and Urdu, preserves the current path on switch (English /en/product/brass-lotus-lamp → Urdu /ur/product/brass-lotus-lamp).

- Build components/layout/SearchOverlay.tsx — full-screen search input with autocomplete-ready structure (the actual search implementation comes in Sprint 4.1).
- Build the shadcn/ui primitives that the above components depend on: Button, Link, Input, Badge, Drawer, Modal, Toast, Sheet, Popover. Configure each to use the design tokens, not shadcn defaults.
- Wire the root layout.tsx with the header, footer, drawer portal, toast portal, and the providers (Zustand, TanStack Query, next-intl).
- Delete the temporary nav menu from Sprint 1.3 — the real Header now provides navigation.

Files created: approximately 25. The key files are the layout components, the shadcn primitives, the providers, and the updated root layout.

Acceptance criteria: mobile and desktop layouts render correctly per the design system; the cart drawer opens and closes from the header trigger; the locale switcher flips dir=rtl on Urdu and back to ltr on English; the mobile menu opens full-screen and closes via swipe or X; the search overlay opens full-screen with a focused input; the header stays sticky on scroll without overlapping content.

» Pause Point 5 — The header and footer are the most-seen components in the entire site. Reviewer should check: typography and spacing match the design system; the gold-on-black contrast is correct; the mobile menu is thumb-friendly; the locale switcher works in both directions; the cart drawer's empty state looks intentional, not broken. Sign off before page bodies are built on top of this shell.

3.3 Sprint 2.3 — Homepage (Days 16-18)

Goal: The homepage is fully built per Chapter 7.1 of the architecture plan, with all six sections rendering with placeholder imagery. This is the highest-visibility page in the entire site and the first impression for every visitor.

Tasks

- Hero section — full-bleed image with the brand promise headline and a primary CTA ('Shop the collection'). Image is a placeholder Pexels photo of a styled living room.
- Featured Categories — three-card grid for Lamps, Plants, and Candles, each card with an image, name, and link to the corresponding /shop/[category] route.

- Bestsellers — product carousel showing 8 products pulled from `productService.list({ sort: 'bestselling' })`, each card with image, name, price, and quick-add button.
- Editorial Story — image plus copy block telling the Aura Living brand story (Light, Life, and Living Beauty), with a link to /about.
- Testimonials — three quote cards with customer name, city, and quote (static data for now; full-stack phase will pull from a reviews table).
- Trust badges row — four badges: 'Cash on Delivery', '7-Day Returns', 'Pakistani Owned', 'Secure Checkout'.
- Newsletter signup — React Hook Form with Zod validation, email field, success state ('Subscribed! Watch your inbox for first look at new arrivals.'), error state for invalid email.

Files created: approximately 12. The key files are `app/[locale]/page.tsx` (composes the sections) and one component per section in `features/home/components/`.

Acceptance criteria: the homepage renders all six sections in order; the layout is responsive (mobile single-column, tablet two-column where appropriate, desktop three-column for the categories grid); all CTAs navigate to the correct routes; the newsletter form validates email and shows success/error states; Lighthouse performance is above 70 on desktop 4G (animations not yet added, so this should be easy).

» Pause Point 6 — The homepage is the brand's first impression. Reviewer should: load it on mobile (375px width) and desktop (1280px); check that all six sections are present and in order; verify the typography and spacing match the design system; confirm the CTAs navigate correctly; check the newsletter form's validation. Sign off before building the PLP and PDP.

3.4 Sprint 2.4 — PLP and PDP (Days 19-23)

Goal: The browse-to-detail flow is complete — a user can filter the product listing, click a card, land on the product detail page, select a variant, and add to cart. This is the critical conversion path and warrants the longest sprint in Phase 2 (5 days).

Tasks

- Build `features/product/components/ProductCard.tsx` — image, name, price, sale badge (-25% pill), quick-add button. Used on PLP, homepage bestsellers, related products, wishlist.

- Build features/product/components/FilterBar.tsx — category checkboxes, price range slider, sort dropdown (Featured, Price low-high, Price high-low, Newest).
- Build app/[locale]/shop/page.tsx and /shop/[category]/page.tsx — product grid with filter state synced to URL query params, pagination, empty state per Chapter 24.
- Build features/product/components/ProductGallery.tsx — main image plus thumbnail strip, swipe carousel on mobile, click-to-change on desktop, image zoom on hover (desktop only).
- Build features/product/components/ProductInfo.tsx — name, price with sale strikethrough, star rating, variant picker (size, colour, scent — varies by product), quantity selector, add-to-cart button, wishlist toggle, trust badges row ('COD Available', '7-Day Returns', 'WhatsApp Support', 'N Happy Customers').
- Build features/product/components/RelatedProducts.tsx — carousel of 4-8 related products from the same category, prefetched via TanStack Query.
- Build features/product/components/ReviewsList.tsx — paginated reviews with TanStack Query, average rating summary at top, 'Be the first to review' CTA when empty.
- Build app/[locale]/product/[slug]/page.tsx — composes the gallery, info, related, and reviews components; fetches product data via productService.getBySlug(); handles not-found, loading, and error states per Chapter 24.
- Add JSON-LD structured data for Product schema on the PDP (Chapter 9.3).
- Add BreadcrumbList JSON-LD on both PLP and PDP.

Files created: approximately 18. The key files are the product components, the PLP and PDP page files, and the JSON-LD components.

Acceptance criteria: a user can apply a filter on the PLP and see the grid update; the URL query string reflects the filter state (so a refresh preserves filters); clicking a product card navigates to the PDP; the PDP gallery supports swipe on mobile and click on desktop; selecting a variant updates the price and add-to-cart state; clicking add-to-cart opens the cart drawer with the new item; the related products carousel shows products from the same category; the reviews list paginates correctly; Lighthouse is above 70 on both PLP and PDP.

» Pause Point 7 — The PLP and PDP are the critical conversion path. Reviewer should: test the filter-and-sort flow on both mobile and desktop; verify the gallery works

on mobile (swipe) and desktop (click + zoom); confirm the variant picker correctly disables out-of-stock variants; test the add-to-cart flow end-to-end; check the empty state when a filter returns no results; verify the breadcrumbs are correct. This is the heaviest review checkpoint in Phase 2.

3.5 Sprint 2.5 — Cart, Checkout, and Confirmation (Days 24-28)

Goal: The full COD-first purchase journey works end-to-end on mock data. A user can add to cart, edit the cart, apply a coupon, check out with COD plus OTP, and land on the order confirmation page. This sprint closes Phase 2 and is the heaviest sprint in the entire plan (5 days, ~22 files).

Tasks

- Build features/cart/store.ts — Zustand store with persist middleware (Chapter 3.4), holding line items, coupon, gift metadata. Persists to localStorage under the 'aura-living-cart' key.
- Build app/[locale]/cart/page.tsx — line items with qty controls and remove buttons, free-shipping threshold nudge (Chapter 22.3), coupon code input (Chapter 22.2), order summary with subtotal, discount, shipping, total.
- Build features/checkout/schemas.ts — the full Zod schema per Chapter 3.6 with the gift extension from Chapter 20.2 and the COD OTP fields. Discriminated union on isGift, phone validation regex for +92 numbers, address fields with city enum.
- Build app/[locale]/checkout/page.tsx — multi-section form: contact (phone, email), shipping address, gift options (collapsible, Ch.20.1), payment method selector (COD default), order summary. React Hook Form with Zod resolver.
- Build features/checkout/components/OtpFlow.tsx — send-OTP button (calls mock service, simulates SMS), 4-digit OTP input, verify button, resend with cooldown timer.
- Build features/checkout/components/PaymentSelector.tsx — radio group with COD (default, selected), JazzCash, Easypaisa, Card, Bank Transfer. Each option shows logo (24x24), name, one-line description. COD requires OTP completion before enabling Place Order.

- Build features/checkout/components/GiftOptionsSection.tsx — collapsible 'This is a gift' checkbox, gift wrap variant picker (Signature Gold, Festive Red, Eco Kraft), handwritten note textarea with 200-char counter, ship-to-different-address toggle that reveals recipient address form.
- Build app/[locale]/orders/[id]/confirmed/page.tsx — order summary with all line items, gift acknowledgment, payment method, delivery address, WhatsApp CTA, 'Continue shopping' button.
- Wire TanStack Query mutations for cart operations (add, update, remove, apply coupon) and for checkout submit (validate, create order, redirect to confirmation).
- Add the conversion-funnel analytics events from Table 25.1: cart.added, cart.viewed, checkout.started, checkout.step_completed, checkout.completed. Wire to the track() helper.

Files created: approximately 22. The key files are the cart store, the cart page, the checkout schema, the checkout page, the OTP flow, the payment selector, the gift options section, and the confirmation page.

Acceptance criteria: a user can add an item to cart, navigate to /cart, edit the quantity, remove an item, apply the WELCOME10 coupon (mock-valid), and see the discount reflected; navigating to /checkout shows the form with COD pre-selected; the OTP flow sends a mock OTP, accepts the mock code 1234, and enables Place Order on success; the gift options section expands when 'This is a gift' is checked; selecting JazzCash or Easypaisa shows the redirect description; submitting the form navigates to the order confirmation page with the correct order summary; the cart persists across a page reload (Zustand persist); Lighthouse is above 70 on cart, checkout, and confirmation.

|| Pause Point 8 — ★ PHASE 2 MAJOR CHECKPOINT. The full purchase journey is now demonstrable. Reviewer should: complete a full COD checkout end-to-end on mobile and desktop; test the OTP flow (send, enter wrong code, enter correct code); test the gift options flow (toggle, select wrap, add note, ship to different address); test the JazzCash and Easypaisa flows (the redirect is mocked but should show the right description); verify the cart persists across reload; check the coupon apply/remove flow; verify all conversion-funnel analytics events fire (visible in the browser dev tools console via the track() helper). Once Phase 2 is signed off, Phase 3 (animations) can begin.

4. Phase 3 — Polish and Animations (Weeks 6-7)

Phase 3 adds the Aura Living signature polish: the GSAP, Framer Motion, and Lenis animations that distinguish a premium site from a generic one. Phase 3 is two sprints over two weeks. The first sprint builds the animation infrastructure (the wrappers and providers that every page will use). The second sprint applies the choreography to every page built in Phase 2. The reduced-motion fallbacks are built into the infrastructure, not bolted on at the end — this is non-negotiable per Chapter 6.5 of the architecture plan.

Phase 3 is the phase where the site starts to feel like Aura Living rather than like a generic e-commerce template. The animations are restrained — no spinning logos, no parallax overload, no auto-playing carousels — but they are choreographed. Every animation has a purpose: to guide attention, to smooth a transition, or to reward interaction. The 80KB animation JS budget (Chapter 6.6) is the hard constraint; any sprint that pushes the animation payload over 80KB must refactor before merging.

4.1 Sprint 3.1 — Smooth Scroll and Parallax Infrastructure (Days 29-31)

Goal: The animation infrastructure is in place — smooth scroll, parallax wrapper, reveal wrapper, magnetic button — all with reduced-motion fallbacks. No page-level choreography is added yet; this sprint builds the tools that Sprint 3.2 will use.

Tasks

- Build `lib/animation/SmoothScrollProvider.tsx` — Lenis smooth scroll, enabled only on fine-pointer devices (Chapter 6.3), disabled on touch and on prefers-reduced-motion. Provider wraps the app in the root layout.
- Build `lib/animation/Parallax.tsx` — GSAP ScrollTrigger wrapper that applies a parallax y-offset to its children based on scroll position. Accepts a speed prop (0 to 1, default 0.3).
- Build `lib/animation/Reveal.tsx` — Framer Motion wrapper that animates children from opacity 0 + y 20 to opacity 1 + y 0 when they enter the viewport (`whileInView`). Accepts a delay prop. Respects prefers-reduced-motion (no animation, instant show).
- Build `lib/animation/MagneticButton.tsx` — Framer Motion wrapper that translates a button towards the cursor on hover (within a 20px radius), springs back on hover-end. Disabled on touch devices.
- Add a global prefers-reduced-motion guard in `styles/globals.css` that disables all CSS transitions and animations when the media query matches.

- Add a `useReducedMotion()` hook (from Framer Motion) and use it in every animation component as a double-check beyond the CSS guard.
- Write a vitest unit test that verifies the `SmoothScrollProvider` does not initialise Lenis when `prefers-reduced-motion` is set, and a Playwright E2E test that verifies the same in a real browser.

Files created: approximately 7. The key files are the four animation components, the global reduced-motion CSS guard, the `useReducedMotion` hook, and the tests.

Acceptance criteria: with `prefers-reduced-motion` unset, Lenis smooth scroll is active on desktop (fine pointer) and inactive on mobile (touch); with `prefers-reduced-motion` set, no animations run at all — the site is fully functional but instant; the Parallax component applies a visible y-offset to a test element on scroll; the Reveal component animates a test element into view; the MagneticButton translates towards the cursor on hover; the total animation JS payload (gsap + framer-motion + lenis combined, tree-shaken) is under 80KB gzipped.

4.2 Sprint 3.2 — Page-Level Choreography (Days 32-36)

Goal: Every page built in Phase 2 is now choreographed with the Aura Living animation language. The site feels premium on desktop and remains smooth on mid-range mobile. This is the longest sprint in Phase 3 (5 days) because it touches every page.

Tasks

- Homepage hero entrance — GSAP timeline that animates the headline (y 40 to 0, opacity 0 to 1), then the subheadline, then the CTA, with 100ms stagger. Triggered on page load.
- Homepage section reveals — wrap each of the six sections in the Reveal component with a 50ms delay between sections so they cascade as the user scrolls.
- Pinned featured-collection section — use GSAP `ScrollTrigger` pin to lock the featured-collection section in place while the user scrolls through the four products, with a horizontal scroll animation.
- PDP gallery image transitions — Framer Motion `AnimatePresence` on the main image so changing the selected thumbnail cross-fades. Use the same ease-aura-living cubic-bezier.
- Cart drawer and mobile menu enter/exit — Framer Motion spring animation (stiffness 300, damping 30) for the slide-in and slide-out. `AnimatePresence` handles exit.

- Testimonial marquee — GSAP infinite timeline that translates the testimonial row from right to left, pauses on hover, resumes on hover-out. Respects reduced-motion (static layout instead).
- 404 and order-confirmation illustrations — Lottie or SVG path animations that draw the illustration on mount (1.2s duration, ease-aura-living).
- View Transitions API on route navigation (Chapter 29.1) — 200ms cross-fade between routes, shared-element morph from PDP image to cart line-item image. Feature-detect at runtime; browsers without support get instant navigation.
- Magnetic button effect on all primary CTAs — the `MagneticButton` wrapper is applied to the homepage hero CTA, the add-to-cart button, the Place Order button, and the newsletter submit.
- Verify the 80KB animation budget is not exceeded. If exceeded, audit which animations can be replaced with CSS or removed entirely.
- Run Lighthouse on all primary routes and confirm no regression from Phase 2.

Files created: approximately 15. The key files are the animation wrappers added to each page, the View Transitions template, the Lottie/SVG illustration components.

Acceptance criteria: the site feels premium and choreographed on desktop (1440px); all animations respect prefers-reduced-motion (verified by toggling the OS setting and reloading); no animation causes visible jank on a mid-range mobile device (test on a 2021-era phone or Chrome DevTools throttling to 4x slowdown); Lighthouse performance score has not regressed from Phase 2; the 80KB animation budget is verified via size-limit.

» Pause Point 9 — ★ PHASE 3 MAJOR CHECKPOINT.
Reviewer should: load the site on desktop and experience the full choreography; toggle prefers-reduced-motion in the OS and reload to verify all animations stop; test on a real mid-range Android device (or Chrome DevTools mid-tier mobile) for jank; verify the View Transitions API works in Chrome (126+) and degrades gracefully in Safari and Firefox; confirm the 80KB animation budget is met. Once Phase 3 is signed off, Phase 4 (SEO and performance) can begin.

5. Phase 4 — SEO and Performance (Week 8)

Phase 4 is the optimisation phase: the metadata, structured data, sitemap, and performance tuning that make the site rank in Google and load fast on Pakistani 3G/4G. Phase 4 is two sprints over one week. The first sprint implements the SEO surface (metadata, JSON-LD, sitemap). The second sprint does the performance audit and bundle tuning. Both sprints are verification-heavy: every change is measured against Lighthouse and the Core Web Vitals targets from Chapter 10.

5.1 Sprint 4.1 — Metadata, Schema, and Sitemap (Days 37-39)

Goal: Every page has correct metadata, JSON-LD structured data is validated, the sitemap is submitted to Google Search Console. This sprint is what makes the site discoverable by search engines.

Tasks

- Configure root metadata in `app/[locale]/layout.tsx` — title template ('%s | Aura Living'), description, Open Graph tags, Twitter Card tags, hreflang alternates, canonical URL strategy.
- Add per-page `generateMetadata` functions to all 27 routes. Each returns title, description, alternates (canonical, hreflang en, hreflang ur, x-default), Open Graph, Twitter.
- Build the JSON-LD components per Chapter 9.3: Organization (on homepage), WebSite (on homepage), Product (on PDP), BreadcrumbList (on PLP and PDP), FAQPage (on /faq), Article (on /journal/[slug]).
- Build `app/sitemap.ts` — dynamically generates the sitemap from the product catalog (mock data for now, Supabase in full-stack phase), includes hreflang alternates for each URL, includes image sitemap entries per Chapter 27.2.
- Build `app/robots.ts` — allows all crawling, references the sitemap URL.
- Build the image sitemap (`sitemap-images.xml`) — lists every product image with caption and license.
- Implement the filtered-PLP canonicalization strategy from Chapter 27.1 — paginated URLs canonicalise to themselves with `noindex` on page 2+, filter-only URLs canonicalise to /shop, category URLs canonicalise to /shop/[category].
- Implement the hreflang strategy from Chapter 18.4 — every page declares en, ur, and x-default alternates.

- Add the accessibility-statement page content (Chapter 27.4) — WCAG 2.2 AA statement with conformance target, audit methodology, known issues, feedback mechanism, last-updated date.
- Validate all structured data with Google's Rich Results Test (manual run by the engineer, screenshots posted to the PR).
- Submit the sitemap to Google Search Console (stakeholder provides credentials).

Files created: approximately 18. The key files are the metadata functions per route, the JSON-LD components, the sitemap, the robots, the image sitemap, the accessibility-statement page.

Acceptance criteria: Google's Rich Results Test passes for the PDP (Product schema), the homepage (Organization + WebSite), the FAQ page (FAQPage), and a journal article (Article); the sitemap is reachable at /sitemap.xml and contains all 27 routes plus hreflang alternates; the image sitemap is reachable at /sitemap-images.xml; Bing Webmaster Tools (optional but recommended) shows the sitemap as submitted; the accessibility statement page renders with correct metadata and is linked from the footer.

5.2 Sprint 4.2 — Performance Audit and Bundle Tuning (Days 40-42)

Goal: Lighthouse performance is 90+ on all primary routes, Core Web Vitals are in the green, the 130KB First Load JS budget is met, and the site loads in under 2 seconds on simulated 4G. This sprint is the final performance gate before launch.

Tasks

- Run Lighthouse on all 27 routes and document the scores in a temp report (a markdown file in the PR). Identify any route below 90.
- Image optimisation pass — ensure every next/image component has explicit width and height (to prevent CLS), priority on the LCP image (hero on homepage, main gallery image on PDP), AVIF and WebP formats via next/image's automatic format negotiation.
- Code-split any route over 130KB First Load JS (Chapter 10.3). Use next/dynamic for below-the-fold components. Verify the cart drawer, mobile menu, and search overlay are dynamically imported (they are not needed on initial load).
- Verify font loading — Inter and Fraunces combined are under 80KB; Noto Nastaliq Urdu loads only when dir=ur (verified by checking network tab in English locale — the Urdu font should not appear).

- Verify animation budget — gsap + framer-motion + lenis combined are under 80KB gzipped (Chapter 6.6). If exceeded, audit and trim.
- Add @vercel/analytics for Core Web Vitals real-user monitoring.
- Add @vercel/speed-insights for field data collection.
- Test on simulated 3G and 4G via Lighthouse throttling. Document the LCP, FCP, TTFB, and TBT for each primary route.
- Verify the security headers from Chapter 17.1 are present on every response (use securityheaders.com or a manual curl -I check).
- Verify the CSP policy blocks an intentionally-injected inline script (add a test script tag to a page, confirm it does not execute).
- Run the axe-core audit on all primary routes and fix any WCAG 2.2 AA violations.

Files created: approximately 5. The key files are the dynamic-import refactors, the Vercel analytics integration, the accessibility fixes, the Lighthouse report markdown.

Acceptance criteria: Lighthouse performance is 90+ on homepage, PLP, PDP, cart, and checkout (mobile and desktop); LCP is at or below 2.0 seconds on simulated 4G; INP is at or below 150ms (measured via the new INP Lighthouse audit); CLS is at or below 0.05; the 130KB First Load JS budget is met on all primary routes (verified via size-limit); the 80KB animation budget is met; all security headers are present and correct; axe-core reports zero violations on all primary routes.

|| Pause Point 10 — ★ PHASE 4 MAJOR CHECKPOINT.
Reviewer should: run Lighthouse independently on the staging URL for homepage, PLP, PDP, cart, and checkout; verify all scores are 90+; check the Rich Results Test for at least one PDP; run the security headers scan; toggle prefers-reduced-motion and verify no animation runs; run axe-core in the browser dev tools on every primary route. Once Phase 4 is signed off, Phase 5 (pre-launch QA) can begin.

6. Phase 5 — Pre-Launch QA (Week 9)

Phase 5 is the final quality assurance phase before the site goes live. The deliverable at the end of Phase 5 is a soft-launched production site (no marketing, monitoring for issues) ready for the official marketing launch in Week 10. Phase 5 is two sprints over

one week. The first sprint hardens the test suite (the 9 critical E2E journeys, the visual regression baselines, the cross-browser and cross-device matrix). The second sprint does the content and legal review and the soft launch.

6.1 Sprint 5.1 — Test Suite Hardening (Days 43-45)

Goal: All 9 critical E2E journeys from Table 16.2 pass on every PR; axe-core reports zero violations on every E2E page; Chromatic visual regression baselines are approved; the cross-browser and cross-device matrix is verified.

Tasks

- Write the 9 critical E2E journeys from Table 16.2 as Playwright test files in tests/e2e/: COD checkout happy path, JazzCash redirect round-trip, cart persistence across reload, search and filter PLP, wishlist add and move to cart, mobile menu navigation, 404 and broken-link recovery, accessibility statement reachability, locale switch to Urdu RTL.
- Run @axe-core/playwright on every E2E page; fix all violations. Document any false positives in an ADR if they cannot be resolved.
- Set up Storybook with one story per component from the Chapter 8 inventory. This is a substantial task — approximately 40 stories.
- Connect Chromatic to the Storybook. Run the first Chromatic build to establish baselines. Reviewer (designer or stakeholder) approves the baselines via the Chromatic UI.
- Run the cross-browser matrix: Chrome (latest), Safari (latest, macOS), Firefox (latest), Edge (latest), Samsung Internet (latest, Android), Safari (latest, iOS). Document any browser-specific issues in the PR.
- Run the cross-device matrix: iPhone SE (375px), iPhone 14 (390px), Samsung Galaxy S22 (360px), iPad Mini (768px), iPad Pro (1024px), MacBook Air (1280px), low-end Android (320px, 2GB RAM, simulated via DevTools). Document any device-specific issues.
- Run the simulated 4G performance test on the full purchase journey. Document the time-to-interactive for each step.

Files created: approximately 50. The bulk is the 9 E2E test files (~80 lines each), the 40 Storybook stories, and the cross-browser/device test report.

Acceptance criteria: all 9 E2E journeys pass on the staging URL; axe-core reports zero violations on every E2E page; Chromatic baselines are approved; the cross-browser matrix shows no critical issues (minor visual differences are acceptable, broken layouts are not); the cross-device matrix shows no critical issues; the simulated 4G performance test shows the full purchase journey completes in under 30 seconds.

6.2 Sprint 5.2 — Content, Legal, and Soft Launch (Days 46-48)

Goal: The site is soft-launched to production (auraliving.pk) with no marketing, monitored for 48 hours for any P0 incidents, ready for the official marketing launch in Week 10. This sprint closes Phase 5 and the frontend build.

Tasks

- Final content review — every page's copy is proofread, every image is checked for correct alt text and correct aspect ratio, every link is clicked (no 404s, no external links to the wrong destination).
- Legal sign-off on the privacy policy, terms of service, and accessibility statement. Stakeholder provides the final legal copy if not already in place.
- Production deploy runbook — a markdown document in docs/Runbook.md covering: how to deploy (merge to main), how to rollback (Vercel dashboard, promote previous deployment), how to flip feature flags (Vercel dashboard), how to activate maintenance mode (Vercel Edge Config flag), monitoring URLs (Vercel Analytics, Sentry, Google Search Console), on-call contact info.
- Configure Sentry for error monitoring (the frontend-only phase uses Sentry's browser SDK; the full-stack phase will add the server SDK).
- Configure Vercel Analytics and Speed Insights for the production deployment.
- Soft launch to production — merge main to the production branch (or trigger the Vercel production deploy). The site goes live at auraliving.pk with no marketing.
- Monitor for 48 hours — watch Vercel Analytics for traffic and Core Web Vitals, watch Sentry for any errors, watch Google Search Console for crawl issues. Document any incidents in an incident log.

- Official marketing launch happens in Week 10. This is out of scope for the frontend build but the frontend team should be on standby for the first 48 hours of marketing traffic to handle any scale or content issues.

Files created: approximately 5. The key files are docs/Runbook.md, the Sentry configuration, the production env vars, the incident log template.

Acceptance criteria: the production site is live at auraliving.pk (or the placeholder domain); Vercel Analytics shows the soft-launch traffic (the team's own visits); Sentry reports zero errors in the first 48 hours; Google Search Console shows the sitemap as submitted and crawling begins; the runbook is reviewed and approved by the engineering team; the marketing team has confirmed they are ready to launch in Week 10.

» Pause Point 11 — ★ GO LIVE. The soft launch is complete. The site is live, monitored, and stable. The decision to flip marketing on (Week 10) is the stakeholder's call. The frontend team is on standby for the first 48 hours of marketing traffic.

7. Pause Points and Decision Gates

This chapter consolidates the eleven pause points from the sprint chapters into a single reference. At each pause point, the engineer stops, posts a summary, and waits for sign-off. The table below lists each pause point, the sprint it closes, the decision the reviewer must make, and the cost of getting it wrong (the downstream impact of a wrong sign-off).

#	Sprint	Decision	Cost of Wrong Sign-Off
1	1.1 Scaffold	Confirm scaffold and dependency versions	Low — easy to refactor later
2	1.2 Design Tokens	Confirm design tokens match the architecture plan	High — every page built on wrong tokens needs restyling
3	1.3 Routes	Confirm route structure	Medium — adding routes later is cheap, removing is messy
4	1.4 CI/CD		

#	Sprint	Decision	Cost of Wrong Sign-Off
		★ Phase 1 sign-off	Very high — foundation problems compound
5	2.2 Shell	Header and footer look right	High — most-seen components
6	2.3 Homepage	Homepage sign-off	High — first impression for every visitor
7	2.4 PLP/PDP	Conversion path sign-off	Very high — direct revenue impact
8	2.5 Checkout	★ Phase 2 sign-off	Very high — full purchase journey
9	3.2 Animations	★ Phase 3 sign-off (feel)	Medium — animations can be toned down later
10	4.2 Performance	★ Phase 4 sign-off (metrics)	Medium — performance can be tuned post-launch
11	5.2 Soft Launch	★ GO LIVE	Critical — production traffic

Table 7.1 — Eleven pause points and their decision gates

7.1 Reviewer Response Times

The plan assumes reviewers respond to pause points within one business day. If a reviewer is unavailable (vacation, travel, deadline on another project), the engineer should be told in advance so they can either pick up an independent sprint (Sprint 2.1 mock data can be done while waiting on Sprint 1.4 sign-off, for example) or pause cleanly. The worst outcome is silence: the engineer waits, then proceeds at their own risk, then the reviewer returns and requests changes that undo three days of work. Communication is the cheapest risk mitigation in this plan.

7.2 Escalation at Major Checkpoints

The five major checkpoints (Pause Points 4, 8, 9, 10, 11) warrant a longer review than the minor pause points. The recommendation is a 30-minute live walkthrough (screen share or in-person) at each major checkpoint, where the engineer demonstrates the deliverable and the reviewer asks questions. This is dramatically more efficient than async review of a written summary, because misunderstandings surface in real time. After the walkthrough, the reviewer sends a written sign-off (or change request) within one business day.

8. Open Questions and Prerequisites

Before Sprint 1.1 begins, six prerequisites must be in place. Five of them require decisions or assets from the stakeholder; the sixth is the architecture plan itself, which is already delivered. This chapter lists each prerequisite, the question that needs an answer, and the impact on the plan if it is missing.

8.1 GitHub Repository

Question: Should I create the GitHub repository `aura-living` under your personal account, or do you have an existing organisation the repo should live under? If the latter, please provide the org name and confirm I have admin access to create repositories and configure GitHub Actions.

Impact if missing: Sprint 1.4 (CI/CD) cannot proceed. Sprints 1.1 through 1.3 can proceed locally but cannot be deployed or collaborated on. Recommended: have the repo ready before Sprint 1.1 begins, even if it is empty — the engineer can push the scaffold as the first commit.

8.2 Domain Registration

Question: Is `auraliveing.pk` registered and pointing at Vercel nameservers? If not, do you want to register it now, or use a placeholder domain (`aura-living.vercel.app`) for staging and production until DNS is ready?

Impact if missing: The site can deploy to Vercel's default subdomain (`aura-living.vercel.app` or similar) for staging. Production launch (Phase 5) requires the real domain. Recommended: register the domain during Phase 1 so DNS propagation completes by Phase 5.

8.3 Vercel Account

Question: Do you have a Vercel account (free or Pro tier) that I should target for the deployment, or should we create a new one? If existing, please confirm I have owner or member access on the team.

Impact if missing: Sprint 1.4 cannot proceed. Vercel is the deployment platform assumed throughout the architecture plan; using a different platform (Netlify, AWS Amplify, self-hosted) would require re-evaluating the PPR, ISR, and Edge runtime decisions. Recommended: Vercel Pro tier for the team (free tier works for development but lacks preview deployments for the marketing team).

8.4 Image Assets

Question: Do you have brand photography ready for Phase 1 and Phase 2, or should we use placeholder imagery (Pexels stock photos) for the entire frontend build and swap to brand photography in Phase 5?

Impact if missing: Placeholder imagery works for the entire frontend build. The architecture plan assumes Pexels placeholders throughout. Brand photography (if available) can be dropped into the same image slots without code changes — the next/image component and the image URLs are decoupled. Recommended: use Pexels for Phases 1–4, plan the brand photography shoot for Phase 5. This unblocks the engineer immediately.

8.5 Brand Wordmark

Question: Should I generate a typographic placeholder wordmark for Phase 1 (the words 'Aura Living' in Fraunces display serif, gold on black), or are you supplying a designed logo before Sprint 1.4?

Impact if missing: A typographic placeholder works for Phases 1–4. The wordmark is just a styled text element in the Header component; swapping to a designed logo (SVG) is a single-file change. Recommended: typographic placeholder for now, designed logo in Phase 5 alongside the brand photography.

8.6 Architecture Plan Access

Prerequisite: The Aura Living Frontend Architecture Plan v1.1 must be accessible to every engineer working on the build. It is the reference document for every decision in this execution plan; every sprint references it by chapter number.

Impact if missing: Engineers will make ad-hoc decisions that contradict the architecture plan, requiring rework. Recommended: share the v1.1 docx with every engineer via a shared drive or git-lfs-tracked file in the repo. The doc lives in /home/z/my-project/download/Aura-Living-Frontend-Architecture-Plan-v1.1.docx.

8.7 Scope for First Build Session

Final question: Do you want me to start with Sprint 1.1 (scaffold) immediately, or wait for the answers to questions 8.1 through 8.5? If you answer 8.1 (GitHub) and 8.3 (Vercel) first, I can begin Sprint 1.1 locally and push to GitHub on day one. The other questions (8.2 domain, 8.4 images, 8.5 logo) can be answered before Sprint 1.4 without blocking the early sprints.

9. Definition of Done per Phase

This chapter consolidates the acceptance criteria from each phase into a single reference. A phase is done when all of its criteria are met, no exceptions. If a criterion cannot be met, the phase is not done — the engineer either completes the criterion or documents the deviation in an ADR and gets explicit stakeholder approval to proceed without it.

9.1 Phase 1 — Foundation Definition of Done

- Next.js 16 project scaffolded with TypeScript strict, Tailwind 4, ESLint, Prettier.
- All runtime and dev dependencies installed per Appendix B of the architecture plan.
- Folder structure matches Chapter 4: app/, features/, components/, lib/, services/, utils/, styles/, tests/.
- Design tokens implemented in styles/tokens.css with the full palette, type scale, spacing, z-index, breakpoints.
- No inline styles anywhere — ESLint rule enforces this.
- Inter, Fraunces, and Noto Nastaliq Urdu fonts loaded via next/font, total under 80KB.
- All 27 routes reachable with locale prefixing (en, ur).
- Middleware handles locale detection, root redirect, and static-asset exclusion.
- GitHub repository created and pushed.
- Vercel project wired: production = main, preview = PRs.
- CI pipeline with four jobs (lint-typecheck, unit-component, e2e-essentials, bundle-size) runs on every PR.
- Staging URL loads the homepage placeholder with correct fonts and colours.
- PR template and ADR directory in place.
- Husky pre-commit hook fires on every commit.

9.2 Phase 2 — Core Pages Definition of Done

- Mock data layer: 48 SKUs, 6 categories, 8 collections, 12 journal articles, 24 reviews.
- TypeScript service contracts defined for every domain (product, cart, checkout, search, review, journal, collection, account, inventory).
- MSW handlers mirror every service contract for component tests.
- Header, footer, mobile menu, cart drawer, locale switcher, search overlay all functional.
- shadcn/ui primitives (Button, Link, Input, Badge, Drawer, Modal, Toast, Sheet, Popover) configured with design tokens.
- Homepage renders all six sections with placeholder imagery and responsive layout.
- PLP with filter bar, product grid, pagination, empty state.
- PDP with gallery, info column, related products, reviews list.
- Cart page with line items, qty controls, free-shipping nudge, coupon input.
- Checkout with contact, shipping, gift options, payment selector (COD default), OTP flow.
- Order confirmation page with summary, gift acknowledgment, WhatsApp CTA.
- Zustand cart store with persist middleware — cart survives page reload.
- TanStack Query for all server state (product, search, reviews).
- React Hook Form + Zod for all forms (newsletter, checkout, contact).
- Conversion-funnel analytics events wired (cart.added, cart.viewed, checkout.started, checkout.step_completed, checkout.completed).
- Lighthouse performance 70+ on homepage, PLP, PDP, cart, checkout, confirmation.

9.3 Phase 3 — Animations Definition of Done

- SmoothScrollProvider (Lenis) active on desktop, disabled on mobile and reduced-motion.

- Parallax component (GSAP ScrollTrigger) with speed prop.
- Reveal component (Framer Motion whileInView) with delay prop, reduced-motion safe.
- MagneticButton component, disabled on touch devices.
- Homepage hero entrance animation (GSAP timeline).
- Homepage section reveals cascade on scroll.
- Pinned featured-collection section on homepage.
- PDP gallery image transitions (Framer Motion AnimatePresence).
- Cart drawer and mobile menu enter/exit spring animations.
- Testimonial marquee (GSAP infinite timeline, pause on hover).
- 404 and order-confirmation illustration draw-on animations.
- View Transitions API on route navigation (200ms cross-fade, shared-element morph PDP-to-cart).
- Magnetic button effect on all primary CTAs.
- All animations respect prefers-reduced-motion (verified by toggling OS setting).
- No animation causes visible jank on mid-range mobile (tested on 2021-era device or DevTools 4x slowdown).
- Lighthouse performance has not regressed from Phase 2.
- Animation JS payload under 80KB gzipped (gsap + framer-motion + lenis combined).

9.4 Phase 4 — SEO and Performance Definition of Done

- Root metadata configured in app/[locale]/layout.tsx (title template, description, OG, Twitter, hreflang).
- Per-page generateMetadata functions on all 27 routes.
- JSON-LD components: Organization, WebSite, Product, BreadcrumbList, FAQPage, Article.
- Dynamic sitemap (sitemap.xml) with hreflang alternates and all 27 routes.
- Image sitemap (sitemap-images.xml) with all product images.
- robots.txt with sitemap reference.

- Filtered-PLP canonicalization strategy implemented (Chapter 27.1).
- hreflang strategy implemented (Chapter 18.4).
- Accessibility statement page with full content (Chapter 27.4).
- Rich Results Test passes for PDP, homepage, FAQ, journal.
- Sitemap submitted to Google Search Console.
- Lighthouse performance 90+ on homepage, PLP, PDP, cart, checkout (mobile and desktop).
- LCP at or below 2.0s on simulated 4G.
- INP at or below 150ms.
- CLS at or below 0.05.
- First Load JS at or below 130KB on all primary routes.
- Animation JS payload at or below 80KB.
- Security headers (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy) present on every response.
- Vercel Analytics and Speed Insights configured.
- axe-core reports zero WCAG 2.2 AA violations on all primary routes.

9.5 Phase 5 — Pre-Launch QA Definition of Done

- All 9 critical E2E journeys pass on staging.
- axe-core reports zero violations on every E2E page.
- Storybook with one story per component (approximately 40 stories).
- Chromatic visual regression baselines approved.
- Cross-browser matrix verified (Chrome, Safari, Firefox, Edge, Samsung Internet, iOS Safari).
- Cross-device matrix verified (iPhone SE, iPhone 14, Galaxy S22, iPad Mini, iPad Pro, MacBook Air, low-end Android).
- Simulated 4G performance test on full purchase journey under 30 seconds.
- Final content review complete (all copy, all images, all links).

- Legal sign-off on privacy policy, terms, accessibility statement.
- Production deploy runbook in docs/Runbook.md.
- Sentry error monitoring configured.
- Production deployment live at auraliving.pk (or placeholder domain).
- 48-hour soft-launch monitoring period complete with zero P0 incidents.
- Marketing team confirmed ready for Week 10 launch.

9.6 Document Version Note

This execution plan is v1.0, dated June 2026, and is the companion to the Aura Living Frontend Architecture Plan v1.1. If the architecture plan is revised (v1.2 or beyond), this execution plan should be reviewed for consistency. The next revision of this execution plan (v1.1) will incorporate any changes arising from senior-developer review of this document and will coincide with the start of Phase 1 implementation.